# Central Monitoring: Docker Deployment Guide

How the dashboard was put into Docker, every file that was created, and how to deploy it on the server PC step by step.

Version: 22 October 2026 · Server: Windows PC with Docker Desktop, office network only

> **Status:** the Docker setup is **built and tested on the development PC**: the backend syncs real Odoo data inside Docker, and the dashboard opens at `http://localhost/` through Nginx. It is ready to be deployed on the server PC with section 6.


1. Docker in plain words
2. How the system is set up
3. How it was built (development PC log)
4. Every file, with explanations
5. Before deploying: checklist
6. Deploy on the server PC, step by step
7. Updating to a new version
8. Daily commands
9. Troubleshooting
10. Safety rules
11. Final deployment checklist


## 1. Docker in plain words

Docker packs an application together with everything it needs (the right Node.js version, the libraries, the code) so it runs the same way on any computer that has Docker. The server PC does not need Node.js installed.

| Word | Meaning | Everyday comparison |
|---|---|---|
| **Image** | A packed, read-only box with the app and everything it needs. Built from a Dockerfile. | An installer file |
| **Dockerfile** | The recipe that says how to build the image, step by step. | A cooking recipe |
| **Container** | A running copy of an image. It can be started, stopped, deleted and recreated from the image at any time. | The installed, running program |
| **Docker Compose** | `docker-compose.yml` lists the containers and their settings, so one command starts everything. | A "start all" shortcut |
| **Port mapping** | `"80:80"` means: port 80 on the PC leads to port 80 inside the container. A container without a port mapping cannot be reached from the network. | A door into the box |
| **.dockerignore** | Files that are never copied into the image (for us: `.env` with the API key). | A "do not pack" list |
| **Nginx** | A small, fast web server. It serves the dashboard files and forwards `/api` requests to the backend. | A receptionist |
| **WSL** | Windows Subsystem for Linux. Containers are small Linux systems; Docker Desktop on Windows needs WSL to run them. | Linux built into Windows |

> **Golden rules:**
> 1. An image is a **snapshot**. After a code change, rebuild: `docker compose up -d --build`.
> 2. Settings in `backend/.env` are read when a container is **created**. After editing `.env`, run `docker compose up -d` (a plain `restart` keeps the old values).

## 2. How the system is set up

```
Office PCs / wall monitors (Chrome)   http://SERVER-IP/
        |
        v
+------------------ Server PC (Docker) -------------------+
|  [ frontend container ]  Nginx, port 80 (open)          |
|      serves the dashboard files                         |
|      forwards /api/...  ---+                            |
|                            v                            |
|  [ backend container ]   Node.js, port 4000 (internal)  |
|      syncs Odoo every 30 s, calculates the KPIs         |
+--------------------------|------------------------------+
                           |  read-only XML-RPC (HTTPS)
                           v
                  Odoo (erp.sawo.com)
```

- Browsers only ever talk to the **frontend** container (port 80). The backend has **no open port**; only Nginx can reach it inside Docker.
- The backend is the only part that talks to Odoo, and it is strictly **read-only**.
- Data is kept **in memory**: there is no database to back up. After a restart, the backend simply syncs again from Odoo (about 30 seconds).

### Files that make up the Docker setup

| File | Purpose |
|---|---|
| `docker-compose.yml` (project root) | Starts both containers, connects them, opens port 80 |
| `backend/Dockerfile` | Recipe for the backend image (Node.js) |
| `backend/.dockerignore` | Keeps `.env` and local files out of the backend image |
| `frontend/Dockerfile` | Recipe for the frontend image (build with Node, serve with Nginx) |
| `frontend/nginx.conf` | Nginx settings: serve the dashboard, forward `/api` |
| `frontend/.dockerignore` | Keeps local files out of the frontend image |
| `backend/.env` (**not** in Git) | Odoo login, API key and settings. Created by hand on each PC from `backend/.env.example` |

## 3. How it was built (development PC log)

This is the record of what was done, in order, including the problems and how they were fixed.

### Step A: Docker Desktop showed "WSL not installed"

Fixed by installing WSL from an **Administrator** PowerShell, then restarting the PC:

```
wsl --install --no-distribution     # WSL only, no Ubuntu (Docker does not need it)
# restart the PC, then check:
wsl --version                       # result: WSL version 3.0.3.0
```

### Step B: Docker checked

```
docker --version          # result: Docker version 29.8.2
docker run hello-world    # result: "Hello from Docker! ... working correctly."
```

### Step C: Backend Docker files created

`backend/Dockerfile`, `backend/.dockerignore` and a first `docker-compose.yml` (backend only, temporary port 4000 for testing). Full contents in section 4.

### Step D: First build failed because the firewall blocked npm

```
npm error code ECONNRESET
npm error network request to https://registry.npmjs.org/xmlrpc/-/xmlrpc-1.3.2.tgz failed
```

| Check | Result | Meaning |
|---|---|---|
| DNS | Works | The name `registry.npmjs.org` is found |
| TCP port 443 | Works | The connection opens |
| HTTPS download | Timed out | The firewall's web filter / HTTPS inspection cuts it |

**Fix:** the firewall admin allowed outbound HTTPS to `registry.npmjs.org` (web-filter whitelist, exempt from SSL inspection). Docker Hub was already allowed.

### Step E: Backend image built and tested

```
docker compose build backend
# added 74 packages ... naming to central-monitoring-backend:latest done  (image: 257 MB)
```

Test: the container was started on a spare port, and `/api/health` and `/api/sync/status` answered `success: true`. Log: `[sync] OK: 1302 orders, 14131 lines (26.6s)`.

### Step F: Frontend Docker files created, backend made proxy-aware

- New: `frontend/Dockerfile`, `frontend/nginx.conf`, `frontend/.dockerignore`.
- `docker-compose.yml`: added the `frontend` service on port 80 and **removed** the backend's open port.
- Backend code: new setting `TRUST_PROXY` (in `config/app.js`, used in `server.js`). Behind Nginx every request seems to come from Nginx; this lets the backend see each browser's real address, so the rate limit (600 requests per minute) counts each screen separately. Docker Compose sets it, so `.env` does not need it. All 58 backend tests still pass.

### Step G: Both containers built and tested together

```
docker compose build      # frontend: added 224 packages ... built in 453ms
docker compose up -d
docker compose ps         # backend Up (4000/tcp, internal) / frontend Up (0.0.0.0:80->80/tcp)
```

| Test | Result |
|---|---|
| `http://localhost/` (dashboard) | 200 OK |
| `http://localhost/sales-orders` (React page, refresh works) | 200 OK |
| `/api/health`, `/api/sync/status`, `/api/dashboard/summary` through Nginx | 200 OK, connection ok, 1,302 orders |
| Backend reachable from the network? | No (correct): only `4000/tcp` inside Docker |

A separate "start automatically after reboot" step was **skipped**: the server PC stays on (see the note in section 6, step 9).

## 4. Every file, with explanations

These are the exact contents. If a file is ever lost, it can be recreated from here.

**docker-compose.yml (project root)**

```
# Central Monitoring: start everything with `docker compose up -d --build` (run in this folder).
# Open the dashboard at http://<this PC's IP>/ (port 80). Only the frontend is reachable from the
network.
services:
  backend:
    build: ./backend
    # Settings and the Odoo API key are read from backend/.env when the container starts (never
copied into the image).
    env_file: ./backend/.env
    environment:
      # Requests arrive through Nginx: trust its X-Forwarded-For so the rate limit sees each
browser's address
      TRUST_PROXY: "true"
    restart: unless-stopped
    # No "ports": the backend is only reachable by the frontend container, not from the network.

  frontend:
    build: ./frontend
    depends_on:
      - backend
    restart: unless-stopped
    ports:
      - "80:80"
```

`restart: unless-stopped` = if a container crashes, Docker starts it again automatically (unless you stopped it yourself).

**backend/Dockerfile**

```
# Backend image: Node.js + only the production libraries + our src/ folder.
# Secrets are NOT baked in: docker-compose passes backend/.env to the container when it starts.
FROM node:24-alpine

ENV NODE_ENV=production
WORKDIR /app

# Install libraries first (this layer is cached until package-lock.json changes)
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY src ./src

# Run as the non-root "node" user that the official image provides
USER node

EXPOSE 4000
CMD ["node", "src/server.js"]
```

**backend/.dockerignore**

```
# Never copy secrets or local installs into the image
.env
.env.*
node_modules
test
scripts
*.log
```

**frontend/Dockerfile**

```
# Frontend image, in two stages:
#   1. "build": use Node to turn the React code into plain HTML/CSS/JS files (npm run build -> dist/)
#   2. final:   a small Nginx web server that serves those files and forwards /api to the backend
# Only stage 2 ends up in the image, so Node and the source code are not shipped.

FROM node:24-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.28-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

**frontend/.dockerignore**

```
# Libraries and the build output are recreated inside the image; env files never go in
node_modules
dist
.env
.env.*
*.log
```

**frontend/nginx.conf**

```
# Nginx inside the frontend container: serves the built dashboard and forwards /api to the backend
container.
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;
  server_tokens off;

  gzip on;
  gzip_types text/css application/javascript application/json image/svg+xml;

  # API -> backend container (the same job the Vite proxy does in development).
  # Docker's internal DNS (127.0.0.11) is asked again every 10 s, so a recreated backend container is
found.
  location /api/ {
    resolver 127.0.0.11 valid=10s;
    set $backend http://backend:4000;
    proxy_pass $backend;
    proxy_set_header Host $host;
    # The real browser address, so the backend's rate limit counts each screen separately
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 120s;
  }

  # Built files have a content hash in their name, so browsers may cache them for a long time
  location /assets/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
  }

  # Dashboard pages (/sales-orders, /settings, ...) are React Router pages: always answer with
index.html.
  # no-cache: browsers check for a new version, so an update shows up on the next reload.
  location / {
    try_files $uri $uri/ /index.html;
    add_header Cache-Control "no-cache";
  }
}
```

**Backend code changes (TRUST_PROXY)**

```
# backend/src/config/app.js  (added inside appConfig)
  // true only behind our Nginx (Docker): then the client address comes from X-Forwarded-For.
  // Keep false when the backend is reachable directly, or clients could fake their address.
  trustProxy: process.env.TRUST_PROXY === 'true',

# backend/src/server.js  (added right after `const app = express();`)
// Behind Nginx (Docker) every request comes from Nginx; trust ONE proxy hop to see the real browser
address.
if (appConfig.trustProxy) app.set('trust proxy', 1);
```

## 5. Before deploying: checklist

| Item | Why |
|---|---|
| Server PC: Windows 10/11 Pro (64-bit), 8 GB RAM or more, 20 GB free disk | Docker Desktop + WSL need it |
| Virtualization enabled in the BIOS | Task Manager → Performance → CPU → "Virtualization: Enabled". Otherwise WSL cannot run |
| Administrator rights on the server PC | To install WSL and Docker Desktop |
| Firewall allows the server to reach `registry.npmjs.org` and Docker Hub | Images download libraries while building (section 3, step D) |
| Server can reach `https://erp.sawo.com` | The backend reads Odoo |
| A fixed IP address for the server PC | Wall monitors and office PCs open `http://SERVER-IP/`; if the IP changes, every screen loses the dashboard |
| Odoo login and API key for the server's `backend/.env` | **Use a truly read-only Odoo user** for production (the current development account can write in Odoo) |
| The latest code, including the Docker files, committed to Git | So the server gets exactly what was tested |

## 6. Deploy on the server PC, step by step

### Step 1: Install WSL (Administrator PowerShell)

Start → type **PowerShell** → right-click **Windows PowerShell** → **Run as administrator**:

```
wsl --install --no-distribution
```

**Restart the PC.** Then, in a normal PowerShell: `wsl --version` must show a version number.

### Step 2: Install Docker Desktop

1. Download Docker Desktop for Windows from `https://www.docker.com/products/docker-desktop/` and install it with the default options (**"Use WSL 2"** checked).
2. Open Docker Desktop and wait until it shows that the engine is running. Signing in is not needed.
3. In Docker Desktop → **Settings** → **General**, turn on **"Start Docker Desktop when you sign in to your computer"**.
4. Check in PowerShell:

   ```
   docker --version
   docker run hello-world      # must print "Hello from Docker!"
   ```

### Step 3: Install Git and get the project

Install Git for Windows ( `https://git-scm.com/download/win` , default options). Then, in PowerShell:

```
cd C:\
git clone https://github.com/KiyoshiGwapo/Central-Monitoring-SAWO.git central-monitoring
cd C:\central-monitoring
```

`git clone` downloads a copy of the project. If the repository is private, Git asks you to sign in to GitHub. **Without Git:** copy the project folder (with a USB drive or network share) to `C:\central-monitoring` , but **leave out** the `node_modules` and `dist` folders and **never copy** `backend/.env` .

### Step 4: Create the server's settings file

```
cd C:\central-monitoring
copy backend\.env.example backend\.env
notepad backend\.env
```

Fill in the values in Notepad and save:

| Setting | Value |
|---|---|
| `ODOO_URL` , `ODOO_DB` | Already filled in ( `https://erp.sawo.com` , `sawo` ) |
| `ODOO_USERNAME` , `ODOO_API_KEY` | The **read-only** Odoo user's login and API key. Type them in; never send them by chat or email |
| `ALLOW_WRITABLE_ACCOUNT` | `false` for a real read-only user. Only set `true` if you knowingly still use the development account |
| `CORS_ORIGIN` | The dashboard address, e.g. `http://172.16.1.50` (optional; only used in a help message) |
| The other settings | Keep the example values (sync every 30 s, timezone Asia/Manila, timeout 60000) |

**Never** commit, email, or share `backend/.env` . It is ignored by Git and by Docker on purpose.

### Step 5: Build and start

```
cd C:\central-monitoring
docker compose up -d --build
```

`--build` builds both images (the first time takes a few minutes: libraries are downloaded), `up -d` starts both containers in the background.

### Step 6: Check that it runs

```
docker compose ps                    # backend and frontend must both say "Up"
docker compose logs -f backend       # wait for "[sync] OK: ... orders", then Ctrl+C (only stops
watching)
```

On the server PC, open `http://localhost/` in Chrome: the dashboard must show real data.

### Step 7: Allow port 80 in Windows Firewall (Administrator PowerShell)

```
New-NetFirewallRule -DisplayName "Central Monitoring (HTTP 80)" -Direction Inbound -Protocol TCP -
LocalPort 80 -Action Allow -Profile Domain,Private
```

This lets other office PCs open the dashboard. `Domain,Private` = office networks only, never public Wi-Fi. (Docker Desktop sometimes adds its own rule; adding this one is still safe.)

### Step 8: Find the server's address and test from another PC

```
ipconfig       # look for "IPv4 Address", e.g. 172.16.1.50
```

On another office PC, open `http://172.16.1.50/` (your server's IPv4 address). Check: real data, the Dashboard / Sales Orders / Settings pages work, **F5** on Sales Orders does not give an error, and the last-sync time in the header updates.

Ask IT to make this IP **fixed** (a DHCP reservation or static IP), so the address never changes.

### Step 9: Wall monitors

On each wall-monitor PC, set up Chrome in kiosk mode pointing at the server address, as described in `docs/Wall_Monitor_PC_Setup.pdf`:

```
"C:\Program Files\Google\Chrome\Application\chrome.exe" --kiosk http://172.16.1.50/
```

> **If the server PC is ever restarted** (for example by Windows Update): Docker Desktop starts when the user signs in (step 2), and both containers start again by themselves ( `restart: unless-stopped` ). After about 30 seconds the dashboard has fresh data again; the wall monitors reconnect on their own.

## 7. Updating to a new version

When the code has changed (new features or fixes):

```
cd C:\central-monitoring
git pull                          # download the latest code (or copy the new files over, keeping
backend\.env)
docker compose up -d --build      # rebuild the images and replace the containers
docker compose ps                 # both "Up"
```

The dashboard is unavailable for a few seconds while the containers are replaced, and needs about 30 seconds to sync again. Wall monitors reload the new version by themselves on their next refresh.

**Only a setting changed** (in `backend\.env`)? Then `docker compose up -d` is enough (no `--build`).

**Free disk space** now and then (old image versions pile up): `docker image prune` (asks before deleting).

## 8. Daily commands

Run in a normal PowerShell, inside the project folder (`cd C:\central-monitoring`).

| Command | What it does |
|---|---|
| `docker compose ps` | Show the containers and their status |
| `docker compose logs -f backend` | Watch the backend output live (Ctrl+C stops watching only) |
| `docker compose logs --tail 100 backend` | Show the last 100 lines |
| `docker compose logs --tail 100 frontend` | Nginx log: every page and API request |
| `docker compose up -d` | Start everything; also applies `.env` changes |
| `docker compose up -d --build` | Rebuild with the latest code, then start |
| `docker compose restart backend` | Restart the backend with the same settings |
| `docker compose down` | Stop and remove the containers (images are kept). **The dashboard goes offline.** |
| `docker images` | List the images on this PC |
| `docker image prune` | Delete unused old images (asks first) |

The same is available in the **Docker Desktop** window → **Containers**: status, logs, start/stop buttons.

### Development PC: Docker and `npm run dev` side by side

The development setup still works as before (`npm run dev` in `backend` and `frontend`, dashboard at `http://localhost:5173`). Docker uses port 80 and does not open port 4000, so both can run at once. But then **two** backends read Odoo every 30 seconds: stop the one you are not using (`Ctrl+C` for npm, `docker compose down` for Docker).

## 9. Troubleshooting

| Problem | Cause and fix |
|---|---|
| "WSL not installed" when opening Docker Desktop | Section 6, step 1: `wsl --install --no-distribution` as Administrator, then restart. |
| "Virtualization not enabled" / error `0x80370102` | Turn on virtualization in the BIOS (ask IT). |
| "Cannot connect to the Docker daemon" / "docker engine not running" | Docker Desktop is not running yet. Open it and wait until it says it is running. |
| `npm error ECONNRESET` or timeouts during a build | The firewall blocks npm. Open `https://registry.npmjs.org/xmlrpc` in Chrome; if it fails, ask IT to whitelist `registry.npmjs.org` (section 3, step D). |
| "Port is already allocated" for port 80 | Another program uses port 80 (for example IIS or another web server). Stop it, or change `"80:80"` to `"8080:80"` in `docker-compose.yml` and use `http://SERVER-IP:8080/`. |
| Works on the server, not from other PCs | Windows Firewall: section 6, step 7. Also check both PCs are on the office network, and the IP is correct ( `ipconfig` ). |
| Backend "Restarting" or "Exited" | It crashes on start. Read `docker compose logs --tail 100 backend`. Usually a missing or wrong value in `backend\.env`. Fix it, then `docker compose up -d`. |
| Dashboard shows "data may be outdated" / connection error | The backend cannot read Odoo (internet down, Odoo down, wrong API key). The dashboard keeps the last good data. Check `docker compose logs --tail 50 backend` for the sync error. |
| Page opens but "502 Bad Gateway" | Nginx is up but the backend is not (yet). Wait 10 s; check `docker compose ps` and the backend log. |
| Code changes do not show | The image is old: `docker compose up -d --build`, then reload the page. |
| `.env` change has no effect | Use `docker compose up -d`, not `restart`. |

## 10. Safety rules

- **The API key never goes into an image or into Git.** `.dockerignore` and `.gitignore` exclude `backend/.env`; Docker hands it to the container only when it starts.
- **Never upload (push) these images** to Docker Hub or any public registry.
- The dashboard has **no login** and shows real company data: keep it on the **office network only**. Never forward port 80 to the internet.
- The backend is **read-only toward Odoo** (only read calls are allowed in the code). For production, use an Odoo user that is **also read-only inside Odoo**.
- Only the frontend (port 80) is reachable from the network. Do not add a `ports:` entry to the backend.

## 11. Final deployment checklist

- ☐ WSL installed, PC restarted, `wsl --version` OK
- ☐ Docker Desktop installed, starts at sign-in, `docker run hello-world` OK
- ☐ Firewall allows `registry.npmjs.org`, Docker Hub and `erp.sawo.com`
- ☐ Project in `C:\central-monitoring` (latest code, includes the Docker files)
- ☐ `backend\.env` created with the **read-only** Odoo user
- ☐ `docker compose up -d --build` done, `docker compose ps` shows both "Up"
- ☐ Backend log shows `[sync] OK`
- ☐ `http://localhost/` on the server shows real data
- ☐ Windows Firewall rule for port 80 added
- ☐ Server IP made fixed by IT
- ☐ `http://SERVER-IP/` works from another office PC (pages, F5, auto-refresh)
- ☐ Wall-monitor PCs point Chrome kiosk at `http://SERVER-IP/`